

1. What is INNER JOIN?

- **INNER JOIN** is used to combine rows from two or more tables.
- It returns only matching records from both tables.
- If a record is present in one table but not present in the other table, it will not appear in the output.

Simple meaning:

INNER JOIN = matching records from both tables only

Example:

customers.customer_id = orders.customer_id

Only those orders will come where the **customer_id** exists in both **customers** and **orders**.

2. When Do We Use INNER JOIN?

Use **INNER JOIN** when we want only valid matched data.

Common use cases:

- Customer orders report
- Product sales report
- Payment report
- ETL data enrichment
- Fact table and dimension table joins
- Valid transaction reporting
- Reporting where unmatched records are not required

Example business requirement:

Show all orders with customer details where the customer exists.

For this, **INNER JOIN** is suitable.

3. Important Point

INNER JOIN removes unmatched records.

Example:

- Customer exists but has no order → not shown
- Order exists but customer does not exist → not shown
- Join key is NULL → not matched
- Product ID does not exist in product table → not shown after product join

So **INNER JOIN** is good for clean matched data, but not good for finding missing records.

For missing/unmatched records, we usually use:

LEFT JOIN

ANTI JOIN

FULL OUTER JOIN

4. Complete SQL Setup

Assume we do not have any table already created.

We will create a new database and all required tables from scratch.

Create Database

```
DROP DATABASE IF EXISTS joins_practice;
```

```
CREATE DATABASE joins_practice;
```

USE joins_practice;

Create customers Table

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50) NOT NULL,  
    city VARCHAR(50),  
    signup_date DATE  
);
```

This table stores customer details.

Create products Table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50)  
);
```

This table stores product details.

Create orders Table

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    order_amount DECIMAL(10,2),  
    order_date DATE,  
    order_status VARCHAR(30)  
);
```

This table stores order details.

Note:

We are not adding foreign key constraints in this practice because we want to insert some invalid records also. This helps us understand how joins behave with dirty data.

Create payments Table

```
CREATE TABLE payments (  
    payment_id INT PRIMARY KEY,  
    order_id INT,  
    payment_mode VARCHAR(30),  
    payment_status VARCHAR(30),  
    paid_amount DECIMAL(10,2),  
    payment_date DATE  
);
```

This table stores payment details.

5. Insert Sample Data

Insert customers

```
INSERT INTO customers
```

```
(customer_id, customer_name, city, signup_date)
```

```
VALUES
```

```
(1, 'Anurag', 'Bengaluru', '2026-01-10'),
```

```
(2, 'Stuti', 'Pune', '2026-02-15'),
```

```
(3, 'Rahul', 'Delhi', '2026-03-12'),
```

```
(4, 'Priya', 'Mumbai', '2026-04-05'),
```

```
(5, 'Neha', 'Bengaluru', '2026-05-20');
```

Insert products

```
INSERT INTO products
```

```
(product_id, product_name, category)
```

```
VALUES
```

```
(201, 'Laptop', 'Electronics'),
```

```
(202, 'Mouse', 'Accessories'),
```

```
(203, 'Keyboard', 'Accessories'),
```

```
(204, 'Monitor', 'Electronics');
```

Insert orders

INSERT INTO orders

(order_id, customer_id, product_id, order_amount, order_date, order_status)

VALUES

(101, 1, 201, 50000.00, '2026-06-01', 'Delivered'),

(102, 1, 202, 1000.00, '2026-06-02', 'Delivered'),

(103, 2, 203, 2500.00, '2026-06-03', 'Delivered'),

(104, 99, 204, 15000.00, '2026-06-04', 'Delivered'),

(105, 3, 999, 3500.00, '2026-06-05', 'Delivered'),

(106, NULL, 201, 45000.00, '2026-06-06', 'Pending'),

(107, 5, 202, 1200.00, '2026-06-07', 'Cancelled');

Important observations:

- Order 101 and 102 belong to Anurag.
- Order 103 belongs to Stuti.
- Order 104 has customer_id = 99, but customer 99 does not exist.
- Order 105 has product_id = 999, but product 999 does not exist.
- Order 106 has customer_id = NULL.
- Order 107 belongs to Neha but has no payment record.

Insert payments

INSERT INTO payments

(payment_id, order_id, payment_mode, payment_status, paid_amount, payment_date)

VALUES

```
(1001, 101, 'UPI', 'Success', 50000.00, '2026-06-01'),  
(1002, 102, 'Card', 'Success', 1000.00, '2026-06-02'),  
(1003, 103, 'UPI', 'Failed', 0.00, '2026-06-03'),  
(1004, 105, 'Wallet', 'Success', 3500.00, '2026-06-05'),  
(1005, 999, 'UPI', 'Success', 3000.00, '2026-06-08');
```

Important observations:

- Payment **1001** is for order **101**.
 - Payment **1002** is for order **102**.
 - Payment **1003** is for order **103**, but payment failed.
 - Payment **1004** is for order **105**.
 - Payment **1005** is for order **999**, but order **999** does not exist.
 - Order **107** has no payment.
-

6. Check Data Before Joins

Always check data before writing joins.

```
SELECT * FROM customers;
```

```
SELECT * FROM products;
```

```
SELECT * FROM orders;
```

```
SELECT * FROM payments;
```

Check row counts:

```
SELECT COUNT(*) AS customer_count FROM customers;
```

```
SELECT COUNT(*) AS product_count FROM products;
```

```
SELECT COUNT(*) AS order_count FROM orders;
```

```
SELECT COUNT(*) AS payment_count FROM payments;
```

Expected counts:

customers = 5

products = 4

orders = 7

payments = 5

This is important because after joining, row counts may change.

7. Basic INNER JOIN Syntax

```
SELECT
```

```
    table1.column_name,
```

```
    table2.column_name
```

```
FROM table1
```

```
INNER JOIN table2
```

```
    ON table1.common_column = table2.common_column;
```

Example:

```
SELECT
```

```
    o.order_id,
```

```
    c.customer_name,
```

```
o.order_amount  
  
FROM orders o  
  
INNER JOIN customers c  
  
ON o.customer_id = c.customer_id;
```

Here:

- `orders` table has order details.
 - `customers` table has customer details.
 - `customer_id` is the common column.
 - Only matching customer IDs will come in output.
-

8. INNER JOIN Between orders and customers

Business question:

Show all valid orders with customer details.

Query:

```
SELECT  
  
o.order_id,  
  
o.customer_id,  
  
c.customer_name,  
  
c.city,  
  
o.order_amount,  
  
o.order_date,
```

o.order_status

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id;

Expected output:

order_id	customer_id	customer_name	city	order_amount	order_date	order_status
101	1	Anurag	Bengaluru	50000.00	2026-06-01	Delivered
102	1	Anurag	Bengaluru	1000.00	2026-06-02	Delivered
103	2	Stuti	Pune	2500.00	2026-06-03	Delivered
105	3	Rahul	Delhi	3500.00	2026-06-05	Delivered
107	5	Neha	Bengaluru	1200.00	2026-06-07	Cancelled

Why only 5 rows?

Orders table has 7 rows.

But:

- Order 104 has customer ID 99, which does not exist in customers.

- Order 106 has NULL customer ID.

So these two records are removed by INNER JOIN.

9. INNER JOIN with products

Business question:

Show customer name, product name and order amount.

Query:

```
SELECT
    o.order_id,
    c.customer_name,
    p.product_name,
    p.category,
    o.order_amount
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id;
```

Expected output:

order_id	customer_name	product_name	category	order_amount
101	Anurag	Laptop	Electronics	50000.00
102	Anurag	Mouse	Accessories	1000.00
103	Stuti	Keyboard	Accessories	2500.00
107	Neha	Mouse	Accessories	1200.00

Why is order 105 missing?

- Order 105 has a valid customer.
- But it has product_id = 999.
- Product 999 does not exist in the products table.
- So when we join with products, order 105 is removed.

Important point:

As we add more INNER JOINS, rows can reduce further.

10. INNER JOIN with payments

Business question:

Show customer name, order amount, payment mode and payment status.

Query:

SELECT

c.customer_name,

o.order_id,

o.order_amount,

pay.payment_mode,

pay.payment_status,

pay.paid_amount

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id

INNER JOIN payments pay

ON o.order_id = pay.order_id;

Expected output:

customer_name	order_id	order_amount	payment_mode	payment_status	paid_amount
Anurag	101	50000.00	UPI	Success	50000.00
Anurag	102	1000.00	Card	Success	1000.00
Stuti	103	2500.00	UPI	Failed	0.00

Rahul	105	3500.00	Wallet	Success	3500.00
-------	-----	---------	--------	---------	---------

Why is order 107 missing?

- Order 107 has a valid customer.
 - But there is no payment record for order 107.
 - INNER JOIN needs matching records in all joined tables.
 - So order 107 is removed.
-

11. INNER JOIN with all four tables

Business question:

Create a final customer order product payment report.

Query:

SELECT

c.customer_name,

c.city,

p.product_name,

p.category,

o.order_id,

o.order_amount,

o.order_status,

pay.payment_mode,

pay.payment_status,

pay.paid_amount

```

FROM orders o

INNER JOIN customers c

    ON o.customer_id = c.customer_id

INNER JOIN products p

    ON o.product_id = p.product_id

INNER JOIN payments pay

    ON o.order_id = pay.order_id;

```

Expected output:

customer_name	city	product_name	category	order_id	order_amount	order_status	payment_mode	payment_status	paid_amount
Anurag	Bengaluru	Laptop	Electronics	101	50000.00	Delivered	UPI	Success	50000.00
Anurag	Bengaluru	Mouse	Accessories	102	1000.00	Delivered	Card	Success	1000.00
Stuti	Pune	Keyboard	Accessories	103	2500.00	Delivered	UPI	Failed	0.00

Why only 3 rows?

- Order 104 has invalid customer.
 - Order 105 has invalid product.
 - Order 106 has NULL customer.
 - Order 107 has no payment.
 - So only orders 101, 102 and 103 match across all tables.
-

12. INNER JOIN with WHERE Condition

Business question:

Show orders placed by Bengaluru customers.

Query:

```
SELECT
    c.customer_name,
    c.city,
    o.order_id,
    o.order_amount
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
WHERE c.city = 'Bengaluru';
```

Expected output:

customer_name	city	order_id	order_amount
Anurag	Bengaluru	101	50000.00
Anurag	Bengaluru	102	1000.00

Neha	Bengaluru	107	1200.00
------	-----------	-----	---------

13. INNER JOIN with Aggregation

Business question:

Find total amount spent by each customer.

Query:

```
SELECT
    c.customer_id,
    c.customer_name,
    SUM(o.order_amount) AS total_spent
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.customer_name;
```

Expected output:

customer_id	customer_name	total_spent
d	e	t

1	Anurag	51000.00
2	Stuti	2500.00
3	Rahul	3500.00
5	Neha	1200.00

Why is Priya missing?

- Priya has no orders.
 - INNER JOIN only returns matching customers with orders.
 - If we need all customers including zero orders, we should use LEFT JOIN.
-

14. INNER JOIN with Multiple Conditions

Sometimes we need more than one condition while joining.

Example:

We want to match city and target month.

Create target table:

```
CREATE TABLE city_targets (  
    city VARCHAR(50),  
    target_month DATE,  
    target_amount DECIMAL(10,2)  
);
```

Insert data:

```
INSERT INTO city_targets  
(city, target_month, target_amount)  
VALUES  
( 'Bengaluru', '2026-06-01', 60000.00),  
( 'Pune', '2026-06-01', 10000.00),  
( 'Delhi', '2026-06-01', 8000.00),  
( 'Mumbai', '2026-06-01', 7000.00);
```

Query:

```
SELECT  
    c.city,  
    SUM(o.order_amount) AS actual_sales,  
    t.target_amount  
FROM customers c  
INNER JOIN orders o  
    ON c.customer_id = o.customer_id  
INNER JOIN city_targets t  
    ON c.city = t.city  
    AND t.target_month = '2026-06-01'  
GROUP BY  
    c.city,  
    t.target_amount;
```

Here we are joining using:

city + target_month

This is common in reporting scenarios.

15. INNER JOIN and NULL Values

Important rule:

INNER JOIN does not match NULL values using normal equality.

Example:

Order 106 has `customer_id = NULL`.

This order will not come in this query:

```
SELECT
    o.order_id,
    o.customer_id,
    c.customer_name
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id;
```

Why?

Because in SQL:

`NULL = NULL`

is not considered true.

NULL means unknown.

So if the join key is NULL, INNER JOIN usually removes that row.

16. INNER JOIN Can Reduce Rows

Check order count:

```
SELECT COUNT(*) AS order_count  
FROM orders;
```

Output:

7

Check count after joining orders with customers:

```
SELECT COUNT(*) AS orders_customers_join_count  
FROM orders o  
INNER JOIN customers c  
    ON o.customer_id = c.customer_id;
```

Output:

5

Check count after joining all four tables:

```
SELECT COUNT(*) AS final_join_count
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id
INNER JOIN payments pay
    ON o.order_id = pay.order_id;
```

Output:

3

Reason:

- INNER JOIN removes unmatched records.
- More INNER JOINS can reduce the row count further.

17. INNER JOIN Can Increase Rows Also

Many students think INNER JOIN always reduces records.

That is wrong.

INNER JOIN can increase rows also if matching keys are repeated.

Create customer offers table:

```
CREATE TABLE customer_offers (
    offer_id INT PRIMARY KEY,
    customer_id INT,
```

```
offer_name VARCHAR(100)
);
```

Insert data:

```
INSERT INTO customer_offers
(offer_id, customer_id, offer_name)
VALUES
(1, 1, 'Festive Discount'),
(2, 1, 'Premium Customer Offer'),
(3, 2, 'UPI Cashback');
```

Now Anurag has:

2 orders

2 offers

Query:

```
SELECT
    c.customer_name,
    o.order_id,
    co.offer_name
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN customer_offers co
```

ON c.customer_id = co.customer_id

WHERE c.customer_id = 1;

Output:

customer_name	order_id	offer_name
Anurag	101	Festive Discount
Anurag	101	Premium Customer Offer
Anurag	102	Festive Discount
Anurag	102	Premium Customer Offer

Why 4 rows?

2 orders x 2 offers = 4 rows

This is called row multiplication.

18. Duplicate Rows After INNER JOIN

Repeated rows after join are not always wrong.

Reasons for repeated rows:

- One customer has many orders.
- One product appears in many orders.

- One order has multiple payment attempts.
- Join key is not unique.
- Many-to-many relationship exists.
- Wrong join condition is used.

Before using **DISTINCT**, always understand the business requirement.

19. What is Grain?

Grain means the level of detail of the output.

Examples:

Customer-level data → one row per customer

Order-level data → one row per order

Payment-level data → one row per payment

Product-level data → one row per product

If business asks for order-level report, one customer can appear multiple times.

That is correct.

If business asks for customer-level summary, we should aggregate first.

20. Correct Approach: Aggregate Before Joining

Business question:

Show one row per customer with total order amount.

Correct query:

```
SELECT
    c.customer_id,
    c.customer_name,
    order_summary.total_amount
FROM customers c
INNER JOIN (
    SELECT
        customer_id,
        SUM(order_amount) AS total_amount
    FROM orders
    GROUP BY customer_id
) order_summary
ON c.customer_id = order_summary.customer_id;
```

This gives one row per customer who has orders.

This is better than blindly using **DISTINCT**.

21. Common Mistakes in INNER JOIN

Mistake 1: Forgetting the ON condition

Wrong:

```
SELECT *
FROM customers c
```

INNER JOIN orders o;

Correct:

SELECT *

FROM customers c

INNER JOIN orders o

ON c.customer_id = o.customer_id;

Mistake 2: Joining on the wrong column

Wrong:

ON c.customer_name = o.customer_id

Correct:

ON c.customer_id = o.customer_id

Mistake 3: Using SELECT *

Avoid:

SELECT *

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id;

Better:

```
SELECT
```

```
    o.order_id,
```

```
    c.customer_name,
```

```
    o.order_amount
```

```
FROM orders o
```

```
INNER JOIN customers c
```

```
    ON o.customer_id = c.customer_id;
```

Reason:

- Output is cleaner.
- Less data is scanned.
- Avoids duplicate column confusion.
- Better for production queries.

Mistake 4: Using DISTINCT without understanding

Wrong mindset:

Rows are repeating, so I will use DISTINCT.

Better mindset:

Why are rows repeating?

What is the grain?

Is this expected because of one-to-many relation?

Mistake 5: Using INNER JOIN when unmatched records are required

If business asks:

Show all customers, including those who never ordered.

Then INNER JOIN is wrong.

Use LEFT JOIN.

22. Practice Questions with Answers

Practice 1

Show all valid orders with customer name and city.

SELECT

o.order_id,

c.customer_name,

c.city,

o.order_amount,

o.order_date

FROM orders o

INNER JOIN customers c

ON o.customer_id = c.customer_id;

Practice 2

Show customer name, product name, category and order amount.

```
SELECT
    c.customer_name,
    p.product_name,
    p.category,
    o.order_amount
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
INNER JOIN products p
    ON o.product_id = p.product_id;
```

Practice 3

Show only successful payments with customer details.

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_amount,
    pay.payment_mode,
    pay.payment_status
```

```
FROM customers c

INNER JOIN orders o

    ON c.customer_id = o.customer_id

INNER JOIN payments pay

    ON o.order_id = pay.order_id

WHERE pay.payment_status = 'Success';
```

Practice 4

Find total revenue generated from each city.

```
SELECT

    c.city,

    SUM(o.order_amount) AS total_revenue

FROM customers c

INNER JOIN orders o

    ON c.customer_id = o.customer_id

GROUP BY c.city;
```

Practice 5

Find number of valid orders placed by each customer.

```
SELECT

    c.customer_name,
```

```
        COUNT(o.order_id) AS total_orders
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_name;
```

Practice 6

Find customers who used UPI payment mode.

```
SELECT DISTINCT
    c.customer_name,
    pay.payment_mode
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN payments pay
    ON o.order_id = pay.order_id
WHERE pay.payment_mode = 'UPI';
```

Practice 7

Create final report with customer, product, order and payment details.

```
SELECT
```

```
c.customer_name,  
c.city,  
p.product_name,  
p.category,  
o.order_id,  
o.order_amount,  
pay.payment_mode,  
pay.payment_status  
FROM orders o  
INNER JOIN customers c  
    ON o.customer_id = c.customer_id  
INNER JOIN products p  
    ON o.product_id = p.product_id  
INNER JOIN payments pay  
    ON o.order_id = pay.order_id;
```

Practice 8

Find total successful paid amount by payment mode.

```
SELECT  
    pay.payment_mode,  
    SUM(pay.paid_amount) AS total_paid_amount  
FROM payments pay  
INNER JOIN orders o
```

```
    ON pay.order_id = o.order_id  
WHERE pay.payment_status = 'Success'  
GROUP BY pay.payment_mode;
```

Practice 9

Find total sales for each product category.

```
SELECT  
    p.category,  
    SUM(o.order_amount) AS total_sales  
FROM orders o  
INNER JOIN products p  
    ON o.product_id = p.product_id  
GROUP BY p.category;
```

Practice 10

Find customer-wise successful payment amount.

```
SELECT  
    c.customer_name,  
    SUM(pay.paid_amount) AS successful_payment_amount  
FROM customers c  
INNER JOIN orders o
```

```
ON c.customer_id = o.customer_id  
INNER JOIN payments pay  
ON o.order_id = pay.order_id  
WHERE pay.payment_status = 'Success'  
GROUP BY c.customer_name;
```

23. Interview Questions and Answers

Q1. What is INNER JOIN?

INNER JOIN returns only matching records from both tables based on the join condition.

Q2. Is JOIN and INNER JOIN same?

In most SQL databases, yes.

If we write only **JOIN**, it behaves like **INNER JOIN**.

Example:

```
FROM customers c  
JOIN orders o  
ON c.customer_id = o.customer_id;
```

This is same as:

```
FROM customers c  
INNER JOIN orders o
```

ON c.customer_id = o.customer_id;

Q3. Can INNER JOIN reduce rows?

Yes.

INNER JOIN can reduce rows because unmatched records are removed.

Example:

Orders table has 7 rows.

After joining with customers, only 5 rows come because two orders do not have valid matching customers.

Q4. Can INNER JOIN increase rows?

Yes.

If join keys are repeated in one or both tables, rows can increase.

Example:

2 matching rows in table A

3 matching rows in table B

Output = $2 \times 3 = 6$ rows

Q5. Why are duplicate rows coming after INNER JOIN?

Repeated rows can come because of one-to-many or many-to-many relationships.

Example:

One customer can have multiple orders.

So the customer will appear once for every order.

This is not always wrong.

Q6. Does INNER JOIN match NULL values?

No.

INNER JOIN does not match NULL values using normal equality.

In SQL:

NULL = NULL

is not true.

NULL means unknown.

Q7. Why did my row count reduce after adding another INNER JOIN?

Because every INNER JOIN requires a matching record in the joined table.

If the record does not match in even one table, it is removed from the final result.

Q8. Should we use DISTINCT to remove duplicate rows after joining?

Not directly.

First understand why rows are repeating.

If rows are repeating because of valid one-to-many relationship, **DISTINCT** may hide the real business logic.

Understand the grain first.

Q9. Does table order matter in INNER JOIN?

For matching output, table order usually does not matter in INNER JOIN.

But if you use `SELECT *`, column order can change.

In production, avoid `SELECT *`.

Q10. When should we not use INNER JOIN?

Do not use INNER JOIN when unmatched records are also needed.

Examples:

- All customers including customers with no orders
- Orders with invalid customer IDs
- Products that were never sold
- Customers with zero payments

Use LEFT JOIN or FULL OUTER JOIN in such cases.

Q11. What is row multiplication in joins?

Row multiplication happens when multiple rows from one table match multiple rows from another table.

Example:

One customer has 2 orders and 2 offers.

Output:

$2 \times 2 = 4$ rows

Q12. What is the biggest beginner mistake in INNER JOIN?

The biggest mistake is not understanding the level of data.

Before joining, always ask:

- Do I need customer-level data?
 - Do I need order-level data?
 - Do I need payment-level data?
 - Is the join key unique?
 - Can one record match multiple records?
-

24. Tricky Interview Scenarios

Scenario 1

You joined 1 lakh customers with 5 lakh orders. After INNER JOIN, output has 8 lakh rows. Is it possible?

Answer:

Yes, it is possible.

INNER JOIN can increase records if the join key has multiple matches.

If one customer has multiple orders and there are multiple matching records in another table, rows can multiply.

Scenario 2

Customer ID 10 exists in both tables, but it is not coming after INNER JOIN. Why?

Possible reasons:

- Data type mismatch
- Extra spaces
- Hidden characters

- Case mismatch
- Wrong join condition
- NULL values
- Different formats

Example:

'10'

'10 '

These may look same, but SQL treats them differently.

Scenario 3

You joined orders with customers and some orders disappeared. Is INNER JOIN wrong?

Answer:

Not always.

INNER JOIN removes unmatched records.

If some orders have invalid customer IDs or NULL customer IDs, they will disappear.

That is expected behavior.

Scenario 4

Business asks for all customers including those who never ordered. Should we use INNER JOIN?

Answer:

No.

INNER JOIN will show only customers who placed orders.

To show all customers, including those with no orders, use LEFT JOIN.

Scenario 5

After joining orders and payments, cancelled orders disappeared. Why?

Possible reason:

Those cancelled orders may not have payment records.

Since INNER JOIN needs matching records in payments also, those orders are removed.

25. Debugging Queries for INNER JOIN

Check row count before join

```
SELECT COUNT(*) AS customer_count FROM customers;
```

```
SELECT COUNT(*) AS order_count FROM orders;
```

Check row count after join

```
SELECT COUNT(*) AS join_count
```

```
FROM orders o
```

```
INNER JOIN customers c
```

```
ON o.customer_id = c.customer_id;
```

Check duplicate customer IDs in orders

```
SELECT
    customer_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Check duplicate product IDs in orders

```
SELECT
    product_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY product_id
HAVING COUNT(*) > 1;
```

Check records after join

```
SELECT
    o.order_id,
    o.customer_id,
    c.customer_name
FROM orders o
```

INNER JOIN customers c

ON o.customer_id = c.customer_id;

26. Quick Revision

- INNER JOIN returns only matching records.
 - It removes unmatched records.
 - It is used for valid matched data.
 - It is common in reporting, ETL, enrichment and analytics.
 - It can reduce rows.
 - It can also increase rows.
 - NULL join keys do not match.
 - Repeated rows are not always duplicates.
 - Understand the grain before using DISTINCT.
 - Use LEFT JOIN when unmatched records are required.
-

27. Final One-Line Summary

INNER JOIN gives only the records that have a match in both joined tables.